Neural Retrieval for Scientific Search: Fine-Tuned Dense and SPLADE Models against BM25 over arXiv Papers and Paragraphs

COS 487 Introduction to Information Retrieval, Course Project Part 2

Mathieu Poulin
University of Southern Maine
USA

Abstract

We extend the Part 1 search system for 34,308 arXiv papers (August–September 2024) with two neural retrievers: a dense bi-encoder (`all-distilroberta-v1`) and a learned sparse model (SPLADE v2), both fine-tuned only on the Train queries with hard negatives and compared against our best Part 1 model, BM25. On the 250 Study queries the dense model is the best system at both levels. At the paragraph level it raises nDCG from 0.415 (BM25) to 0.529 and lenient MRR from 0.428 to 0.561, and SPLADE reaches 0.487 and 0.515; both gains over BM25 are significant on every metric ($p < 0.001$), and at this level dense also significantly beats SPLADE on nDCG and on all precision metrics. At the paper level, where documents are only a title and abstract, the gains are small: dense improves nDCG over BM25 (0.403 vs. 0.360, $p = 0.03$), but no other paper-level difference is significant. The neural models help most where BM25 failed in Part 1: on Hard (paraphrased) queries, paragraph MRR rises from 0.005 to 0.151 with the dense model. The remaining failures are mostly heavily paraphrased Hard queries that no model solves, plus queries that hinge on exact identifiers, numbers or names, where BM25 is still better. With the GPU indexes, a paragraph search takes 8 ms (dense) or 103 ms (SPLADE) instead of 1.7 s.

1 Introduction

Scientific search asks a system to connect short natural-language queries to long, technical documents. Part 1, joint work with N. Borelli, J. Odulio and B. Roy, built the traditional baseline: TF-IDF and BM25 implemented from scratch over one tokenizer and one set of inverted indexes. This part extends that code base. BM25 was the better model, significantly so for paragraph retrieval, but both models nearly failed on Hard queries, which paraphrase the vocabulary of the relevant documents: on those queries BM25's paragraph MRR was 0.005. This part replaces exact term matching with learned representations, which is aimed directly at that vocabulary mismatch.

Collection and tasks. The collection contains 34,308 arXiv papers from August and September 2024, provided as JSON records with `paper_id`, `title`, `abstract` and `paragraphs` (2,275,574 paragraphs in total). *Paper retrieval* returns paper IDs and *paragraph retrieval* returns paragraph IDs; for each query the system returns the top 100 results in 6-column TREC format. `Train.json` has 2,000 queries with paper and paragraph qrels, `Study.json` 250 queries with qrels, and `Test.json` 250 queries without qrels. Each file is 30% easy, 50% medium and 20% hard. We fine-tune only on Train queries; Study and Test queries are never used for training, model selection or tuning. Relevance is 2 (fully relevant), 1 (partially relevant), and 0 for anything not in the qrels. The Study qrels are sparse: 1.56 judged papers and 2.91 judged paragraphs per query on average.

Contributions. (1) Cleaned document text and Train-only fine-tuning data with hard negatives; (2) a fine-tuned dense bi-encoder with exact GPU search; (3) a fine-tuned SPLADE model served from an inverted index of learned term weights; (4) a comparison of both against BM25 on every required metric, with paired significance tests between all three models; (5) an analysis of where neural retrieval helps and where it still fails; (6) an extended interface (neural models, LaTeX rendering, annotation) and a manually labeled Test qrel file for the best model.

2 Methodology

2.1 Part 1 baseline: BM25

BM25 is our best Part 1 model and the baseline for all comparisons here. All text passes through one tokenizer, which strips HTML markup, lowercases, extracts alphabetic tokens with `\b[a-z]+\b` and removes NLTK's 198 English stop words, with no stemming. Papers are indexed by title and abstract (34,308 documents) and paragraphs individually (2,275,574 documents). We use the Okapi formulation

$$\text{score}(q,d) = \sum_{t \in q} \ln\!\left(1 + \frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5}\right) \cdot \frac{\text{tf}_{t,d}(k_1+1)}{\text{tf}_{t,d} + k_1\left(1 - b + b\,|d|/\overline{|d|}\right)},$$

with untuned defaults $k_1 = 1.5$ and $b = 0.75$ and the non-negative IDF variant [1]. Each distinct query term counts once, only documents with a positive score are ranked, and ties are broken by document ID. The BM25 result files are the ones submitted in Part 1.

2.2 Pre-processing for the neural models

The neural encoders use their own subword tokenizers, so preprocessing is limited to the data noise we measured in Part 1:

- **Title metadata.** About 12% of titles have template text appended ("Preprint:", "Thanks:" with author e-mails, "Conference:", "DOI:", "CCS:"). We cut each title at the first such marker, never leaving a title empty.
- **Markup and whitespace.** HTML entities are unescaped, stray tags removed and whitespace collapsed.
- **Fragments.** Paragraphs under 25 characters (737 of 2,275,574, e.g. "Proof." or equation residue) are left out of the neural paragraph indexes. We measure length in characters because some extracted paragraphs lost all their spaces. None of the dropped paragraphs is judged relevant in the Train or Study qrels.

- **Boilerplate is kept.** The 11,478 copies of repeated boilerplate (conference checklist answers, disclosure statements) stay in the index: some Train queries ask about exactly that content, such as negative societal impacts, and have checklist copies judged fully relevant.

Paper-level documents are "title. abstract", the same unit as in Part 1; paragraph-level documents are single paragraphs. Inline LaTeX is left as is, since subword tokenizers handle it better than our Part 1 tokenizer did.

2.3 Fine-tuning data

The 2,000 Train queries are split 90/10 *by query* into 1,800 training and 200 validation queries, so no validation query is seen in training. Every judged document (label 1 or 2) in `Train_paper_qrel.tsv` and `Train_paragraph_qrel.tsv` is a positive, and both levels are mixed into one training set, so a single model per architecture serves both tasks. Each positive is paired with one *hard negative*:

- for a paragraph positive, another paragraph of the same paper that is not judged relevant: topically close, so the model must learn finer distinctions than random negatives teach;
- for a paper positive, a non-relevant paper ranked 10–50 for the query by the *untuned* dense model. We skip the top 10, where unjudged but relevant papers are most likely.

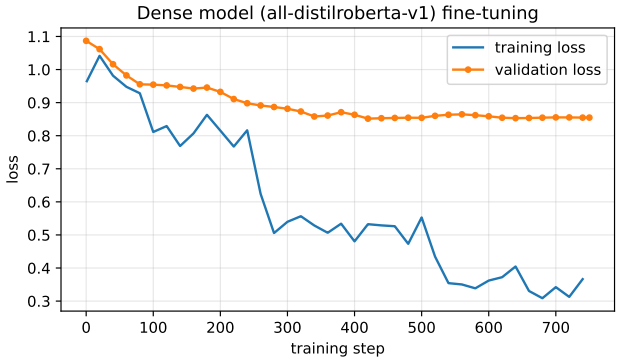
This gives 7,972 training and 922 validation triples (query, positive, negative). One judged Train document is missing from the cleaned corpus and is skipped.

2.4 Dense retrieval

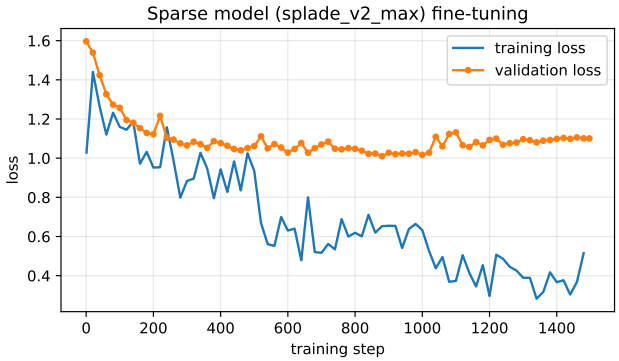
The dense model is the Sentence-Transformers bi-encoder `all-distilroberta-v1` [2]: a 6-layer DistilRoBERTa that mean-pools token embeddings into one 768-dimensional vector per text, compared by cosine similarity. We fine-tune it with the multiple-negatives ranking loss [3]: for each query in a batch, the positive must outscore its hard negative and every other document in the batch, which provides $2B-1$ negatives per query at batch size B. Inputs are truncated to 256 subword tokens. After fine-tuning, every document is encoded once into a unit-length float16 vector; the paragraph index is a $2{,}274{,}837 \times 768$ matrix (3.5 GB). Search is exact: the query vector is multiplied by the whole matrix on the GPU and the top 100 are selected, so no approximate-search error is introduced.

2.5 Sparse retrieval: SPLADE

The sparse model is SPLADE v2 (`naver/splade_v2_max`) [4]. It runs BERT's masked-language-model head over the input and max-pools $\log(1+\mathrm{ReLU}(\cdot))$ over the tokens, producing one non-negative weight for each of the 30,522 vocabulary terms. Most weights are zero, so each text becomes a sparse bag of weighted terms. Unlike BM25, the model can give weight to terms that do not occur in the text (expansion) and can down-weight terms that do. We fine-tune with the SPLADE loss: a sparse multiple-negatives ranking loss on the same triples plus FLOPS regularization [5], weighted 3×10^{-5} for documents and 5×10^{-5} for queries, which penalizes dense vectors and keeps the index sparse. At indexing time we keep the 256 highest-weighted terms per document; documents average



(a) Dense (`all-distilroberta-v1`)



(b) SPLADE (`splade_v2_max`)

Figure 1: Training and validation loss per step (logged every 20 steps; step 0 is the untuned model). One epoch is 250 steps for the dense model and 499 for SPLADE. The kept checkpoints are those with the lowest validation loss: step 420 (dense) and step 880 (SPLADE).

246 non-zero terms at the paper level and 219 at the paragraph level. The vectors are stored as a document × vocabulary matrix in compressed sparse column format, which is exactly an inverted index: one posting list of (document, weight) pairs per term. A query is scored by reading only the posting lists of its non-zero terms and summing query weight × document weight.

2.6 Training procedure

Both models train for 3 epochs with AdamW, learning rate 2×10^{-5}, 10% linear warm-up and mixed precision on one NVIDIA RTX A5000 (24 GB); batches never contain the same text twice, so duplicates cannot act as false negatives. Table 5 lists the settings. Training and validation loss are logged every 20 steps, and the checkpoint with the lowest validation loss is kept. Figure 1 shows both curves. Fine-tuning took 9 minutes for the dense model and 17 minutes for SPLADE; encoding the paragraph collection took about 18 and 40 minutes respectively.

2.7 System and interface

One retrieval interface serves all models: `search(query, model, level, k)` returns BM25 results from the Part 1 indexes and neural results from the GPU indexes in the same format, so the experiment runner, the evaluation script and the Flask interface treat all models alike. The interface keeps its Part 1 features (Figure 2): highlighted snippets, filters by submission month and paper length, search history, and full-text paper pages. The model toggle now also offers Dense and SPLADE whenever their indexes are available. Because 60% of paragraphs contain inline LaTeX, result snippets, paper pages and the annotation page now render math with KaTeX; query-term highlighting skips formulas, and snippets are widened so they never cut a formula in half. The annotation page can run in a lightweight mode that loads only the papers it needs, instead of the 2.4 GB BM25 index.

2.8 Evaluation protocol

Metrics are computed per query and averaged over the 250 Study queries: nDCG and nDCG@5 with linear gain (ideal ranking from all judged documents); MRR; Precision@5; and Precision@10. MRR and Precision@5 (and also Precision@10) are reported in two variants: lenient (relevance ≥ 1) and strict (relevance $= 2$). Unjudged documents count as non-relevant, and Precision@k always divides by k. We compare every pair of models (dense, SPLADE and BM25) with a two-sided paired t-test and a paired randomization (sign-flip) test with 10,000 permutations on per-query differences [6]; $p < 0.05$ is marked as significant.

3 Experimental Results

3.1 Effectiveness

Table 1 gives the results. The dense model is the best system on every metric at both levels. At the paragraph level the gains are large: nDCG rises from 0.415 (BM25) to 0.529 (dense, +28%) and 0.487 (SPLADE, +17%), and lenient MRR from 0.428 to 0.561 and 0.515. Dense ranks a relevant paragraph first for 117 of 250 queries, against 92 for BM25 and 113 for SPLADE, and it leaves only 37 queries with no relevant paragraph in the top 100, half as many as BM25 (74; SPLADE 57). At the paper level the picture is flatter: dense improves nDCG from 0.360 to 0.403 (+12%), while SPLADE (0.378) is close to BM25 and slightly below it on nDCG@5 and P@5. Precision is low for all models because the qrels are sparse: even a perfect ranking would reach a mean lenient P@5 of only 0.307 for papers and 0.553 for paragraphs.

3.2 Significance

Table 2 lists the paired tests; the t-test and the randomization test agree on every comparison. *Part 2 vs. Part 1:* at the paragraph level both neural models beat BM25 on all eight metrics with $p < 0.001$; dense has the higher nDCG on 136 queries and BM25 on 55 (59 ties). At the paper level only one of the sixteen neural–BM25 comparisons is significant: dense nDCG ($p = 0.030$; dense ahead on 92 queries, BM25 on 60). *Dense vs. SPLADE:* at the paragraph level dense is significantly better on nDCG ($p = 0.012$) and on all four precision metrics ($p \leq 0.029$), but not on nDCG@5, MRR_2 or $\text{MRR}_{\geq 1}$ (the last narrowly, $p = 0.056$). Dense has the higher nDCG on 108 queries and SPLADE on 70, so per query the two models trade wins. At the paper level no difference between the Part 2 models is significant. We therefore claim that both neural models outperform BM25 for paragraph retrieval and that dense is the best model overall; for paper retrieval only the dense model's nDCG gain is supported.

3.3 Efficiency

Table 4 reports the average search time per query. A single paragraph query takes 8 ms with the dense model and 103 ms with SPLADE, against 1.7 s for pure-Python BM25 on a laptop CPU in Part 1; the hardware differs, so the comparison shows practical latency rather than algorithmic cost. Dense search is one matrix–vector product on the GPU and grows only linearly with collection size. SPLADE scoring runs on the CPU over the posting lists of the query's terms; expansion gives each query many terms with long posting lists at the paragraph level, so it is slower than dense search there, but still interactive. The single-query time for dense paper search (20 ms) is higher than for paragraphs because it was measured first and includes GPU warm-up; batched, both levels take under 2 ms per query. The price of this speed is the offline work: fine-tuning (9 and 17 minutes), encoding 2.27 million paragraphs (about 18 and 40 minutes) and a GPU with enough memory to hold the 3.5 GB dense index.

3.4 Training behavior

Both validation curves (Figure 1) fall steeply in the first epoch, from 1.087 to 0.898 (dense, step 240) and from 1.596 to 1.052 (SPLADE, step 480), and then flatten: the best validation losses, 0.852 at step 420 and 1.010 at step 880, both occur in the second epoch. The training loss keeps falling and drops in visible steps at each epoch boundary (steps 250 and 500 for dense), the signature of the model memorizing triples it has seen before. For SPLADE the validation loss rises again in the third epoch (to 1.10), so the model began to overfit our 1,800 training queries; keeping the best-validation checkpoint discards those later steps. SPLADE's curves are noisier because its batch is half the size and its loss includes the FLOPS penalty, which changes as the vectors become sparser.

4 Result Analysis

4.1 By query difficulty

Table 3 breaks the results down by the Easy/Medium/Hard labels. The neural gains are concentrated on the queries that defeated BM25. On Hard queries, paragraph MRR rises from 0.005 (BM25) to 0.151 (dense) and paper MRR from 0.090 to 0.232; on Medium paragraph queries nDCG@5 rises from 0.352 to 0.492. On Easy queries, which share many words with their relevant documents, the models are much closer: SPLADE is the best paragraph model (nDCG@5 0.659 vs. 0.624 dense and 0.591 BM25), and at the paper level BM25 is slightly ahead (0.383 vs. 0.363). SPLADE behaves like a better BM25: it helps most where exact terms already match, but improves little on Hard queries (paragraph MRR 0.057). The dense model is the only one that substantially handles paraphrase. Hard queries remain the weak spot for every model: 23 of the 25 queries for which no model retrieves any relevant paragraph in the top 100 are Hard.

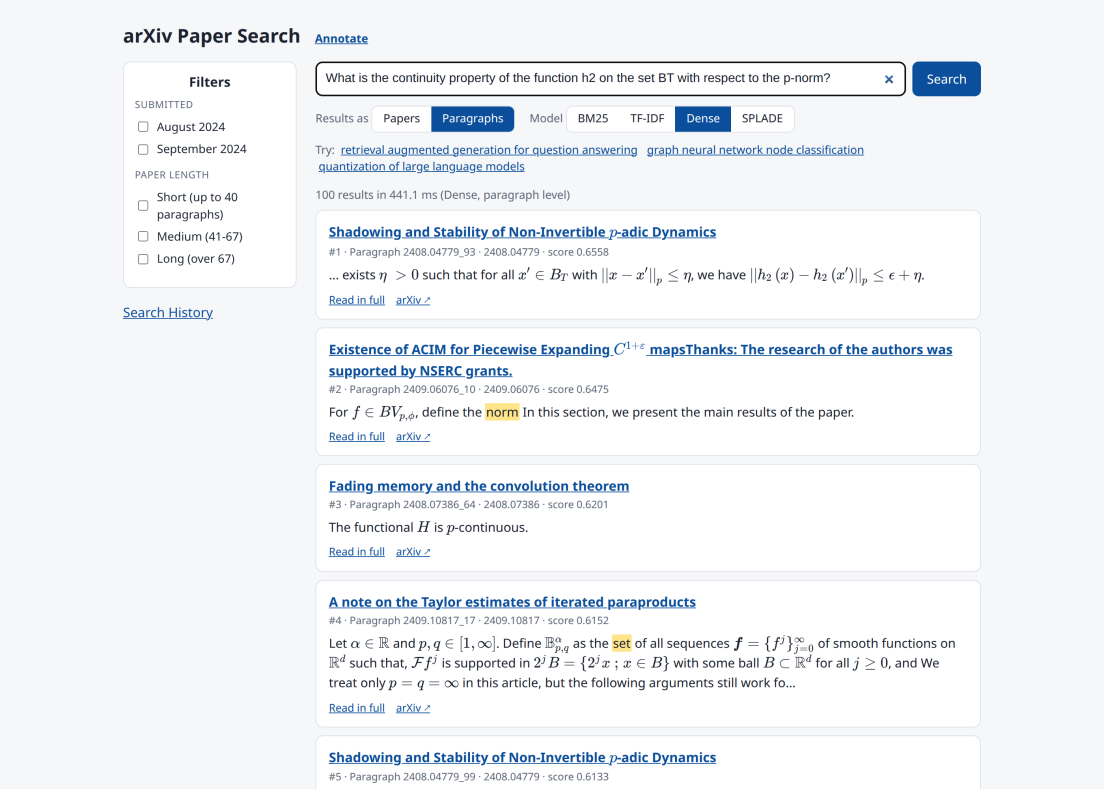


Figure 2: The search interface with the dense model at the paragraph level: filter panel and search-history link (left), query box with level and model toggles (top), and results with highlighted query terms and rendered LaTeX. The query is Study query 2085; the rank-1 paragraph is the judged relevant one, which BM25 does not retrieve. (The reported time includes GPU warm-up for the first query.)

Table 1: Results on the 250 Study queries (mean over queries; ≥ 1 = Relevant or Partially Relevant counted as relevant, 2 = Relevant only). Best value per level in bold.

Model	nDCG	nDCG@5	$\text{MRR}_{\geq 1}$	MRR_2	$\text{P@5}_{\geq 1}$	P@5_2	$\text{P@10}_{\geq 1}$	P@10_2
Paper level								
BM25 (Part 1)	0.3596	0.3119	0.3318	0.3096	0.0888	0.0776	0.0512	0.0432
Dense	**0.4025**	**0.3350**	**0.3597**	**0.3356**	**0.0928**	**0.0832**	**0.0520**	**0.0456**
SPLADE	0.3784	0.3109	0.3337	0.3101	0.0864	0.0744	0.0512	0.0428
Paragraph level								
BM25 (Part 1)	0.4148	0.3536	0.4275	0.3557	0.1472	0.0928	0.0888	0.0528
Dense	**0.5291**	**0.4493**	**0.5610**	**0.4704**	**0.2080**	**0.1320**	**0.1280**	**0.0748**
SPLADE	0.4867	0.4282	0.5150	0.4450	0.1856	0.1176	0.1128	0.0656

4.2 Where the neural models succeed

The clearest wins are the vocabulary-mismatch failures we analyzed in Part 1. Query 2011 asks why heart rhythm abnormalities are harder to identify with *wrist-worn optical sensors* than with *electrical cardiac monitoring*. BM25 ranked a heart-rate-monitor paper first; the dense model ranks first the relevant paper, *Multiclass Arrhythmia Classification using Smartwatch Photoplethysmography Signals*, mapping "wrist-worn optical sensors" to smartwatch photoplethysmography and "heart rhythm abnormalities" to arrhythmia. Query 2009 ("gaze, body positioning, hand motions" with a "digital helper") no longer loses to a hand-manipulation dataset: dense ranks first the relevant study of nonverbal behavior with voice assistants. Query 2101 asks for the "primary guidelines that control the development of material failure" in a "variational framework for cracks"; only the dense model connects this to phase-field fracture and ranks the relevant paper first. In total, 8 paper-level and 17 paragraph-level queries on which BM25 found nothing in the top 100 have a relevant document in the dense model's top 2.

Table 2: Paired significance tests on the 250 Study queries: difference in means, two-sided paired t-test p_t and paired randomization test p_r (10,000 sign flips). * = $p < 0.05$.

	Dense − BM25			SPLADE − BM25			Dense − SPLADE		
Metric	Diff	p_t	p_r	Diff	p_t	p_r	Diff	p_t	p_r
Paper level									
nDCG	+0.043	.030*	.028*	+0.019	.250	.247	+0.024	.159	.161
nDCG@5	+0.023	.297	.296	−0.001	.958	.960	+0.024	.239	.237
$MRR_{\geq 1}$	+0.028	.223	.221	+0.002	.926	.925	+0.026	.225	.225
MRR_2	+0.026	.230	.225	+0.001	.975	.975	+0.025	.210	.211
$P@5_{\geq 1}$	+0.004	.536	.617	−0.002	.687	.787	+0.006	.303	.364
$P@5_2$	+0.006	.328	.403	−0.003	.528	.635	+0.009	.094	.130
$P@10_{\geq 1}$	+0.001	.821	.907	0.000	1.000	1.000	+0.001	.793	.899
$P@10_2$	+0.002	.415	.500	0.000	.870	1.000	+0.003	.287	.358
Paragraph level									
nDCG	+0.114	<.001*	<.001*	+0.072	<.001*	<.001*	+0.042	.012*	.011*
nDCG@5	+0.096	<.001*	<.001*	+0.075	<.001*	<.001*	+0.021	.257	.256
$MRR_{\geq 1}$	+0.134	<.001*	<.001*	+0.088	<.001*	<.001*	+0.046	.056	.053
MRR_2	+0.115	<.001*	<.001*	+0.089	<.001*	<.001*	+0.025	.266	.255
$P@5_{\geq 1}$	+0.061	<.001*	<.001*	+0.038	<.001*	<.001*	+0.022	.018*	.023*
$P@5_2$	+0.039	<.001*	<.001*	+0.025	<.001*	<.001*	+0.014	.020*	.029*
$P@10_{\geq 1}$	+0.039	<.001*	<.001*	+0.024	<.001*	<.001*	+0.015	.007*	.008*
$P@10_2$	+0.022	<.001*	<.001*	+0.013	<.001*	<.001*	+0.009	.007*	.008*

Table 3: Mean nDCG@5 and lenient MRR by query difficulty (Study queries; number of queries in parentheses). B = BM25, D = Dense, S = SPLADE.

		nDCG@5			$MRR_{\geq 1}$		
Level	Difficulty	B	D	S	B	D	S
Paper	Easy (75)	**0.383**	0.363	0.359	**0.410**	0.406	0.378
Paper	Medium (125)	0.365	**0.368**	0.365	0.382	0.383	**0.396**
Paper	Hard (50)	0.073	**0.211**	0.103	0.090	**0.232**	0.111
Paragraph	Easy (75)	0.591	0.624	**0.659**	0.691	0.744	**0.764**
Paragraph	Medium (125)	0.352	**0.492**	0.447	0.438	**0.616**	0.549
Paragraph	Hard (50)	0.003	**0.079**	0.035	0.005	**0.151**	0.057

Table 4: Average search time per Study query (ms) for top-100 retrieval, excluding index loading. BM25: one query at a time, pure Python on a laptop CPU (Part 1). Neural models: query encoding plus search on one NVIDIA RTX A5000, one query at a time and in batches of 50.

	BM25	Single query		Batched	
Level	(CPU)	Dense	SPLADE	Dense	SPLADE
Paper	70.0	19.9	4.7	1.4	3.4
Paragraph	1682.9	8.4	102.6	0.6	94.1

SPLADE's successes are more lexical. For query 2034 ("a last-in-first-out data structure" in Bitcoin), BM25 ranked a generic Bitcoin history paragraph first; SPLADE ranks first a relevant paragraph stating that Bitcoin clients validate transactions by executing scripts, and dense finds a relevant paragraph at rank 2. SPLADE also wins on exact rare terms: for query 2015 about "a single Brent's algorithm iteration", SPLADE ranks the relevant paper first, while the dense model returns a paper on extremes of stochastic processes.

Table 5: Fine-tuning setup. Both models train on the same 7,972 triples from 1,800 Train queries and validate on 922 triples from the 200 held-out Train queries; the checkpoint with the lowest validation loss is kept.

	Dense	SPLADE
Base model	`all-distilroberta-v1`	`splade_v2_max`
Batch size / learning rate	32 / 2e-05	16 / 2e-05
Epochs / steps	3 / 750	3 / 1497
Validation loss: untuned	1.087	1.596
Validation loss: best (step)	0.852 (420)	1.010 (880)

4.3 Failure cases

No model retrieves any relevant document in its top 100 for 48 paper-level queries (10 Easy, 17 Medium, 21 Hard) and 25 paragraph-level queries (2 Medium, 23 Hard). We traced the errors to five causes.

- **Exact identifiers and named entities.** Dense vectors summarize topics and blur exact strings. Query 2232 compares the evaluations of "LSLM" and the "NatLan framework" with Llama-2 (7B); BM25 ranks a relevant paper first by matching the names, but dense returns *Open Llama2 Model for the Lithuanian Language* and SPLADE an LLM-pruning paper. Only four queries have a relevant document in BM25's top 2 but none in the dense top 100 (2015, 2232, 2056, 2175), and all four depend on exact strings: a named algorithm, two system names, a specific technical phrase and a number.

- **Numbers and specific details.** Query 2175 asks for "the average and median relative differences … across the 12 realizations". BM25 ranks the relevant paragraph first. Dense and SPLADE both stay in the right paper but rank neighboring paragraphs first (an overview of formation rates; a table note), because numeric details barely affect the representations; SPLADE still finds the relevant paragraph at rank 4, dense not at all. Query 2056 (inner solid bodies versus outer fluid bodies) fails the same way: dense returns a two-particle quantum mechanics paragraph that shares the word "interaction".
- **Abstract paraphrase.** Some Hard queries are paraphrased so far that no model bridges them. Query 2012 asks about "precision drops when calculating mathematical models on rounded spatial limits", meaning numerical methods on curved mesh boundaries. No model finds either relevant paper; dense returns a paper on numerical integration in physics-informed neural networks, which is at least in the right area.
- **Paper-level representation.** Paper documents contain only a title and abstract, so evidence in the body is invisible at the paper level. This explains why paper-level gains are small for all models: a query about a detail of the method section cannot be matched no matter how good the encoder is. It also explains why paragraph scores exceed paper scores for every model.
- **Truncation and length.** Inputs are cut to 256 subword tokens. Paper-level documents and long paragraphs lose their tail, and LaTeX-heavy text spends many tokens on markup.

4.4 Complementarity and incomplete judgments

The three models retrieve quite different documents: on average only 17–23% of the top 10 papers and 20–26% of the top 10 paragraphs are shared between any two models. Together with the per-query win counts above, this suggests that a fusion of the rankings would outperform each model alone. The low overlap also matters for evaluation. The qrels list only judged relevant documents, and a document that no model in the original judgment pool retrieved is counted as non-relevant even when it answers the query. The dense model's rank-1 paragraph is a judged relevant paragraph for 47% of Study queries, yet in our Test annotation (Section 4.5) 73% of its rank-1 paragraphs were at least partially relevant. Part of that gap is the different query sets, but it suggests that the Study scores underestimate the neural models, which find unjudged relevant documents.

4.5 Test-set annotation

For the best model on the Study queries, the dense model, we labeled the rank-1 paragraph of each of the 250 Test queries on the 3-point scale (`Poulin_qrel_part2_test.tsv`). 112 paragraphs (45%) fully address the query, 70 (28%) provide useful context only, and 68 (27%) are not relevant. By difficulty, the rank-1 paragraph is at least partially relevant for 85% of Easy, 72% of Medium and 56% of Hard Test queries, and fully relevant for 60%, 43% and 26%. The same pattern as on Study appears: partial relevance is common because a paragraph often states the topic of a question without answering every part of it, and many multi-part queries ("… and how do these approaches compare") can be fully answered only by combining paragraphs. **[TODO: Annotation provenance: 19 of these 250 labels were assigned by the author and 231 were assigned with an AI assistant (Claude), applying the scale above to the query and paragraph text. The assignment asks for manual labels: either review and correct the AI-assigned labels in the annotation page before submission, or keep this disclosure in the paper. Delete this note once resolved.]**

4.6 Data noise

The data noise documented in Part 1 affects the neural models differently. Title metadata no longer reaches the paper index, and fragments are removed. Repeated boilerplate is kept on purpose, since some queries ask about it. Display equations removed during the dataset's extraction still leave broken sentences ("Let and where …"), which the encoders read as they are. Inline LaTeX, by contrast, is now an advantage rather than noise: subword tokenizers keep symbols such as B_T and $\|\cdot\|_p$, so for query 2085 (the continuity of a function h_2 on a set B_T under a p-norm) the dense model ranks the relevant paragraph first, while BM25, whose tokenizer drops these symbols, finds nothing.

4.7 Strengths and weaknesses

Dense is the best model overall: it is the only one that handles paraphrase and the fastest at query time, and a single model serves both levels. Its weaknesses are exact identifiers and numbers, an opaque score that cannot be explained term by term, a large GPU-resident index, and the need for training data and a GPU. *SPLADE* keeps the interpretability and the inverted-index structure of BM25, because every score is a sum over vocabulary terms, and it is the best paragraph model on Easy queries; but it improves little on Hard queries, its index is larger than BM25's, and it overfit our small training set quickly. *BM25* needs no training and remains competitive at the paper level, especially on Easy queries and exact names, but it is slow in our implementation and fails on paraphrase. None of the models answers multi-part questions that need evidence from several paragraphs.

5 Conclusion and Future Work

We fine-tuned a dense bi-encoder and a SPLADE model on the Train queries only and compared them with our Part 1 BM25 baseline. Both neural models significantly outperform BM25 for paragraph retrieval on every metric, with the dense model best (nDCG 0.529 vs. 0.415), and the gains are largest on the medium and hard queries that defeated lexical matching. At the paper level, where documents are only a title and abstract, the gains are small and only dense nDCG is significantly better. Neural search is also two orders of magnitude faster per paragraph query on a GPU. Future work follows from the analysis: (1) hybrid retrieval that fuses BM25, SPLADE and dense rankings (e.g. reciprocal rank fusion), since the models retrieve largely different documents and fail on different queries; (2) a cross-encoder or LLM re-ranker over the fused top 100, the focus of Part 3; (3) richer paper representations, such as the maximum or mean over

a paper's paragraph scores instead of the abstract alone; (4) longer inputs and more training data, e.g. synthetic queries generated for unjudged paragraphs; (5) tuning SPLADE's sparsity weights and stopping on the Train validation split; and (6) better handling of LaTeX and of the equations lost during extraction.

References

[1] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.

[2] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proc. EMNLP-IJCNLP*. 3982–3992.

[3] Matthew Henderson, Rami Al-Rfou, Brian Strope, Yun-Hsuan Sung, László Lukács, Ruiqi Guo, Sanjiv Kumar, Balint Miklos, and Ray Kurzweil. 2017. Efficient Natural Language Response Suggestion for Smart Reply. arXiv:1705.00652.

[4] Thibault Formal, Benjamin Piwowarski, Carlos Lassance, and Stéphane Clinchant. 2021. SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval. arXiv:2109.10086.

[5] Biswajit Paria, Chih-Kuan Yeh, Ian E.H. Yen, Ning Xu, Pradeep Ravikumar, and Barnabás Póczos. 2020. Minimizing FLOPs to Learn Efficient Sparse Representations. In *Proc. ICLR*.

[6] Mark D. Smucker, James Allan, and Ben Carterette. 2007. A Comparison of Statistical Significance Tests for Information Retrieval Evaluation. In *Proc. CIKM*. 623–632.